

CS 551 Systems Programming

Lecture 1: Your First Program

The guessing game, and the five ideas it drags in

Where we are

Lecture 0 got you a toolchain, a project, and `hello world`. That is enough to compile something and not enough to learn anything from.

This lecture builds one small program end to end: the guessing game from *The Rust Programming Language*, chapter 2. The point is not the program.

Five decisions other languages let you avoid

Twenty lines of Rust force all of them:

1. Mutability, and why it is opt-in.
2. `Result`, and what Rust does instead of exceptions.
3. Dependencies, and how you find out what one offers.
4. Type inference, and exactly where it stops.
5. Shadowing, and why it is not just reassignment.

Every one of those gets a lecture of its own later. Today you meet them at work.

The guessing game

From *The Rust Programming Language*, chapter 2

What we're building

The computer picks a secret number. The user guesses. We tell them higher or lower until they get it.

It is a small program that touches a lot of Rust fast:

1. Variables, mutability, and shadowing.
2. Reading from standard input.
3. `Result`, and what Rust does instead of exceptions.
4. Adding and using an external dependency.
5. Type inference and where it stops.

Step 1: a variable to read into

```
1 println!("Guess the number");  
2  
3 let mut guess = String::new();  
4  
5 println!("Please input your guess.");
```

Step 1: a variable to read into

```
1 println!("Guess the number");
2
3 let mut guess = String::new();
4
5 println!("Please input your guess.");
```

let creates a variable and **binds** it to whatever is on the right of the =

Step 1: a variable to read into

```
1 println!("Guess the number");
2
3 let mut guess = String::new();
4
5 println!("Please input your guess.");
```

mut marks it mutable. Leave it off and this binding can never change

Step 1: a variable to read into

```
1 println!("Guess the number");
2
3 let mut guess = String::new();
4
5 println!("Please input your guess");
```

String::new() is an **associated function**: it belongs to the **String** type, not to any particular string

Step 1: a variable to read into

```
1 println!("Guess the number");
2
3 let mut guess = String::new();
4
5 println!("Please input your guess.");
```

The `::` is how you say that. A `.` would mean "call this on a value I already have"

Everything is immutable by default

This is the part that surprises people coming from C or Python.

```
1 fn main() {  
2     let x = 5;  
3     x = 6;  
4 }
```

```
error[E0384]: cannot assign twice to immutable variable `x`
```

You cannot change a binding's value once assigned unless you asked for that ability with `mut`. The compiler enforces it, so it is **impossible to change a value by accident**.

Wanting to mutate is fine. Saying so is the requirement.

Step 2: reading a line

```
1 use std::io;
2
3 fn main() {
4     println!("Guess the number");
5
6     let mut guess = String::new();
7
8     println!("Please input your guess.");
9
10    io::stdin()
11        .read_line(&mut guess)
12        .expect("Failed to read line!");
13
14    println!("You guessed {guess}");
15 }
```

Step 2: reading a line

```
1 use std::io;
2
3 fn main() {
4     println!("Guess the number");
5
6     let mut guess = String::new();
7
8     println!("Please input your guess.");
9
10    io::stdin()
11        .read_line(&mut guess)
12        .expect("Failed to read line!");
13
14    println!("You guessed {guess}");
15 }
```

Brings the `io` module into scope, so we can write `io::stdin()` instead of `std::io::stdin()`

Step 2: reading a line

```
1 use std::io;
2
3 fn main() {
4     println!("Guess the number");
5
6     let mut guess = String::new();
7
8     println!("Please input your guess.");
9
10    io::stdin()
11        .read_line(&mut guess)
12        .expect("Failed to read line");
13
14    println!("You guessed {guess}");
15 }
```

Brings the `io` module into scope, so we can write `io::stdin()` instead of `std::io::stdin()`

`read_line` appends to the string you hand it rather than returning a new one, so it needs to modify our `guess`

Step 2: reading a line

```
1 use std::io;
2
3 fn main() {
4     println!("Guess the number");
5
6     let mut guess = String::new();
7
8     println!("Please input your guess.");
9
10    io::stdin()
11        .read_line(&mut guess)
12        .expect("Failed to read");
13
14    println!("You guessed {guess}");
15 }
```

The **&** passes a **reference**: access to the value without handing over ownership of it

A reference is a pointer the compiler is watching.
Lecture 4 is entirely about this.

Step 2: reading a line

```
1 use std::io;
2
3 fn main() {
4     println!("Guess the number");
5
6     let mut guess = String::new();
7
8     println!("Please input your guess.");
9
10    io::stdin()
11        .read_line(&mut guess)
12        .expect("Failed to read");
13
14    println!("You guessed {guess}");
15 }
```

And the `mut` is a second, separate permission: you may *write* through this reference

Two sigils, two ideas. `&` is "you may look," `mut` is "you may write".

`read_line` also returns a `Result`

`read_line` fills in your string *and* returns a value describing whether it worked.

```
1 enum Result<T, E> {  
2     Ok(T),  
3     Err(E),  
4 }
```

`Result` is an **enum** with two variants. Reading from a terminal can fail: the stream can be closed, the bytes can be invalid UTF-8.

`.expect(msg)` says: if this is `Ok`, give me the value inside; if it is `Err`, crash the program with `msg`.

If you ignore the `Result` entirely, the compiler warns you. Rust does not let errors be silent by accident.

A warning about `expect`

`expect` is a fine way to learn, and it is what the book uses in chapter 2.

It is **not** how you handle errors in real code, and the projects in this course explicitly forbid it:

```
1 #![deny(clippy::unwrap_used, clippy::expect_used)]
```

Every `expect` is a decision to crash rather than to handle. That is acceptable in a fifteen-line teaching program and unacceptable in a library.

We cover what replaces it (`?`, `Result` in your own signatures, custom error types) in the error-handling lecture. Until then: `expect` while learning, and know you owe the compiler an answer later.

Step 3: we need a random number

The standard library does not ship a random number generator. We add one as a **dependency**, which in Rust is called a **crate**.

```
1 cargo add rand
```

That edits `Cargo.toml` for you:

```
1 [dependencies]
2 rand = "0.10.2"
```

Cargo honors **semantic versioning**: `"0.10.2"` means "at least 0.10.2, and anything API-compatible with it." `Cargo.lock` records exactly what you actually built against, so your build is reproducible.

Using the dependency

```
1 use std::io;
2
3 fn main() {
4     println!("Guess the number");
5
6     let secret_number = rand::random_range(1..=100);
7
8     println!("The secret number is: {secret_number}");
9
10    let mut guess = String::new();
11
12    println!("Please input your guess.");
13
14    io::stdin()
15        .read_line(&mut guess)
16        .expect("Failed to read line!");
17
18    println!("You guessed {guess}");
19 }
```

About that range

`1..=100` is an **inclusive range**: 1 through 100, with 100 included. `1..100` is exclusive and stops at 99.

Printing the secret number is a debugging move, not a feature. Take it out once the game works.

Note on crate APIs. `rand` changed this API more than once. Older tutorials show `rand::thread_rng().gen_range(1, 101)`, which needs `use rand::Rng;` and does not compile against current `rand`. This is normal. The fix is always the same: read the docs for the version you actually have.

How do you know what a crate provides?

```
1 cargo doc --open
```

Builds documentation for your project **and every one of your dependencies**, from their source, at the exact versions you are using, then opens it in a browser.

This is the single most underused command in the toolchain. It beats searching the web, because the web gives you whatever version was current when the blog post was written.

Also: <https://docs.rs> hosts the same generated docs for every published crate.

READ THE DOCS.

Step 4: compare the guess to the secret

Naive attempt:

```
1 use std::io;
2 fn main() {
3     let secret_number = rand::random_range(1..=100);
4
5     let mut guess = String::new();
6     io::stdin()
7         .read_line(&mut guess)
8         .expect("Failed to read line!");
9
10    if guess == secret_number {
11        println!("You got it right!");
12    } else if guess > secret_number {
13        println!("You guessed too high!");
14    } else {
15        println!("You guessed too low!");
16    }
17 }
```

Step 4: compare the guess to the secret

Naive attempt:

```
1 use std::io;
2 fn main() {
3     let secret_number = rand::random_range(1..=100);
4
5     let mut guess = String::new();
6     io::stdin()
7         .read_line(&mut guess)
8         .expect("Failed to read line!");
9
10    if guess == secret_number {
11        println!("You got it right!");
12    } else if guess > secret_number {
13        println!("You guessed too high!");
14    } else {
15        println!("You guessed too low!");
16    }
17 }
```



Will it compile?

The compiler's objection

```
error[E0277]: can't compare `String` with `{integer}`
|
|       if guess == secret_number {
|           ^^ no implementation for `String == {integer}`
|
= help: the trait `PartialEq<{integer}>` is not implemented for `String`
```

`guess` is a `String`. `secret_number` is an integer. Rust will not compare them, and it will not silently convert one to the other.

Rust has **no implicit numeric conversions** and **no implicit conversion between unrelated types**. If you want to compare a string to a number, *you* say how.

(Rust does have a few coercions, notably `&String` to `&str`. They are narrow, they never lose information, and we meet them when we cover `Deref`.)

Aside: Rust has type inference

We never wrote a type annotation. The compiler worked them out.

```
1 let x = 5;           // integer
2 let y = String::new(); // String, because String::new() returns one
3 let z = x + 1;       // integer, because x is
```

Static typing does not mean you annotate everything. C makes you annotate; that is a limitation of C, not a property of static typing.

Inference does not weaken the checking. Every type is pinned down at compile time; you just did not have to type it out.

You can always annotate explicitly, and sometimes you must:

```
1 let x: i32 = 5;
```

Step 5: parse the string into a number

```
1 use std::io;
2
3 fn main() {
4     let secret_number = rand::random_range(1..=100);
5
6     let mut guess = String::new();
7     io::stdin()
8         .read_line(&mut guess)
9         .expect("Failed to read line!");
10
11     let guess: i32 = guess.trim().parse().expect("You need to give me a number...");
12
13     if guess == secret_number {
14         println!("You got it right!");
15     } else if guess > secret_number {
16         println!("You guessed too high!");
17     } else {
18         println!("You guessed too low!");
19     }
20 }
```

Three things happened on that line

```
1 let guess: i32 = guess
2   .trim()
3   .parse()
4   .expect("You need to give me a number...");
```

Three things happened on that line

```
1 let guess: i32 = guess
2     .trim()
3     .parse()
4     .expect("not a number");
```

trim() strips the newline
read_line kept. "42\n" is not a
number

Three things happened on that line

```
1 let guess: i32 = guess
2   .trim()
3   .parse()
4   .expect("...");
```

parse() converts a string slice into some other type. It can fail, so it returns a **Result**

Three things happened on that line

```
1 let guess: i32 = guess  
2   .trim()  
3   .parse()  
4   .expect("You need t
```

And this annotation is load-bearing. `parse` can produce many types; the `: i32` is how the compiler picks one

Without it, this line does not compile at all. `parse` has no way to guess what you wanted.

Why `: i32` is not decoration

Delete the annotation and the program stops compiling: `parse` has no way to know what you wanted.

Change it to `f64` and `"3.14"` becomes a valid guess. Change it to `u8` and `300` becomes a runtime error rather than a number.

The annotation is doing three jobs at once. It picks the return type of `parse`, it decides which inputs are errors, and it fixes the type of `guess` for the comparison two lines later.

And it shadows the old `guess`

That new `let` does not mutate the old `guess`. It creates a **new** variable that happens to reuse the name, and the old one becomes inaccessible.

```
1 let guess = "42";           // &str
2 let guess: i32 = 42;        // different variable, different type, same name
```

Shadowing is deliberate and idiomatic. It is how you convert a value through a pipeline of types without inventing `guess_str`, `guess_trimmed`, `guess_num`.

Note the difference from `mut`: `mut` changes the value in an existing binding and cannot change its type. Shadowing makes a new binding and can.

What's still missing

The program plays exactly one round.

To loop until the user is right, we need `loop`, `break`, and a better way to branch than `if/else if/else`. The idiomatic version uses `match` on an `Ordering`, and it handles a bad guess without crashing:

```
1 match guess.cmp(&secret_number) {  
2     Ordering::Less      => println!("Too small!"),  
3     Ordering::Greater   => println!("Too big!"),  
4     Ordering::Equal     => break,  
5 }
```

That is the next lecture: enums, pattern matching, and why `match` is more than a `switch`.

Recap

1. **Bindings are immutable by default.** `mut` is opt-in and explicit.
2. **`Result` is a value, not an exception.** `expect` crashes on `Err`; real code handles it.
3. **Crates are dependencies.** `cargo add` plus `cargo doc --open`.
4. **Type inference is real static typing.** Annotate when the compiler cannot decide, or when you want to constrain it.
5. **Shadowing** makes a new binding with an old name, and may change the type.
6. **`expect` is a placeholder.** Every one you wrote today is a `match` you have not written yet.

Exercises

1. Get the guessing game running end to end on your own machine. Remove the line that prints the secret number.
2. Feed it garbage. Type `abc`, then type `3.14`, then hit Ctrl-D. Read each panic message. Which of them is `parse` and which is `read_line`?
3. Change `let guess: i32` to `let guess: u8` and guess `300`. Explain the error in one sentence.
4. Delete `.trim()`. Predict what happens before you run it, then run it.
5. Run `cargo clippy` on your guessing game. Fix anything it complains about.
6. Run `cargo doc --open` and find `rand`'s documentation for `random_range`. What is its return type, and how does the compiler decide it here?

Reading

- **The Rust Programming Language**, chapter 2 (Programming a Guessing Game). Free at <https://doc.rust-lang.org/book/>.
- **The Cargo Book**, the chapter on dependencies: <https://doc.rust-lang.org/cargo/>
- **Rust by Example**, "Hello World" and "Primitives".

Next lecture: enums and pattern matching, which is how you replace every `expect` you wrote today.

Next: Lecture 2

Enums and Pattern Matching

`Option`, `Result`, and why `match` is more than a `switch`.